CS 6210 Spring 2025 Test 3

(120 min Canvas Quiz)

[Total Points: 68]

Table of Contents

Internet Computing [26 points] 1
  Giant Scale Services [10 points] 2
  Map Reduce [7 points] 3
  CDN [9 points] 4
Realtime & Multimedia [14 points] 6
  TS Linux [4 points] 6
  PTS [10 points] 6
Failures & Recovery [10 points] 8
  RioVista [6 points] 8
  Quicksilver [4 points] 9
Security [5 points] 9
  AFS [5 points] 9
HIDDEN - Potpourri [13 points] 10
  LRVM [10 points] 10
  AFS [3 points] 11

Internet Computing [26 points]

Giant Scale Services [10 points]

1. [6 points] You are the Site Reliability Engineer at a leading ML model serving company. Your service is bound by Service Level Agreement that guarantees 99.99% availability (measured by uptime) to your customers. Reviewing the incident logs for the past year, you observe that the service experiences a critical crash on average once every 10 weeks. When a crash occurs, your current automated recovery script takes 10 minutes to detect the failure and restore the service. Management is also considering a stricter internal goal of 99.999% for the next quarter.

a) [3 points] [True/False] Does the current system configuration meet the SLA of 99.99%? Give justification for your answer

Answer – True [1 point]

MTTR = 10 minutes

MTBF – 1 incident every 10 weeks = 10 * 7 * 24 * 60 = 100,800 minutes

Uptime = (MTBF-MTTR)/MTBF

= (100800-10)/100800 = 0.9999007

So, the system is available for 99.99% based on the uptime calculation

2 points for calculation

b) [1 point] [True/False] Given the current system configuration, determine whether it can achieve an SLA of 99.999% availability. Justify your reasoning

Answer – False, as the uptime is 99.99%. The system cannot be available for 99.999%. [No marks given for incorrect reasoning]

c) [2 points] What should be maximum repair time, to meet SLA of 99.999%.

0.99999 = 100800-MTTR/100800

100798.992 = 100800 – MTTR

MTTR = 1.008 minutes

The maximum repair time should be brought down from 10 minutes to 1.008 minutes.

2. [4 points] A search engine cluster can process up to 100,000 QPS. The cluster has 1000 nodes, and the entire corpus of data needed for

processing the queries is partitioned evenly on all the nodes. Under normal load, it can run at 80,000 QPS. Suddenly, a rack switch failure causes 50 compute nodes to go offline instantly. The Load Manager detects this and continues to send traffic to the remaining nodes. The system is designed to return best effort results. If a partition is down, it returns results from the available partitions rather than returning an error.

a) [2 points] Calculate Yield and Harvest immediately after failure

Yield = Queries completed/queries offered

80,000/80,000

Yield = 1

Harvest = available data/complete data

= 1000-50/1000

= 950/1000

Harvest = 0.95

b) [2 points] If the 50 failed nodes are brought back online and the cluster is brought back to its original state of 1000 active nodes. But a breaking news event causes a massive spike in traffic. Now you have to handle 120,000 QPS. The Load Manager is configured to drop (reject) any requests exceeding the maximum capacity to prevent the servers from crashing. Calculate the Yield and harvest in this case?

Ans: yield = 100,000/120,000

= 5/6

= 0.83

Harvest = 1000/1000 = 1

Map Reduce [7 points]

1. [2 points] A junior engineer proposes optimization to the MapReduce library to lower latency for real-time applications. Their proposal is:

"Currently, reduce tasks wait until all Map tasks are complete. We should change this so Reduce workers start executing the user's Reduce function as soon as the first batch of Map tasks are finished, and their data is retrieved."

Give one pro and con of this proposal

Ans:

Pro: Could reduce the time for each Reduce task since they are all not waiting for ALL the mappers to finish.

Con: Could result in no significant reduction in time of each Reduce tasks since they must integrate their partial results with the new intermediate results from the mappers as they come in.

2. [2 points] Network bandwidth is described in the Map-Reduce paper as a "relatively scarce resource". To address this, you decide to implement a feature where all intermediate output from Map tasks is compressed using a data compression algorithm (ignore all compression cost) before being stored on disk and sent to Reduce workers.

(Answer True/False with justification – No points without justification) Are intermediate Map data typically suitable for this type of compression?

Ans:

- [+2] True, since keys are repeated in map files, compression will remove the duplicates.
- [+1] True, but incorrect justification

3. [3 points] "The MapReduce implementation in the paper relies on the underlying file system's atomic rename operation to ensure that the final output is consistent, even when redundant reduce tasks' run simultaneously with primary tasks."

Imagine you are porting MapReduce to a cloud object store like Azure Blob Storage or AWS S3, which does not support atomic renames.

Explain the steps (bullet points) to simulate this atomic rename between the Master and the Workers?

Ans:

- [+1] Worker writes a temporary file and after closing the file, the Worker sends file location to Master.
- [+0.5] Master accepts the first completed response for a task
- [+0.5] Master ignores the any other responses for the same task
- [+1] Master then copies/moves the accepted files to the output location.

CDN [9 points]

[9 points]

In the following problemCD, assume that

- Put(x, y) denotes putting the key-value pair (key = x and value = y)
- Get(x) denotes getting the value corresponding to the key = x
- Assume a Coral system with node-ids 1, 2, 5, 10, 20, 40, 80, 160, 200 in which the "l" value for a node is set to 1; "β" is infinite.
- For simplicity, assume that all puts/gets in this problem using key-based routing go through node-ids: 20, 40, 80, 160, 200 irrespective of the source node of the request.

Bob just created a new video featuring him playing the drums. He wants to use Coral CDN to share it with his buddies.

1. [3 points] To do this, Bob issues a command Put(200, 10). What do 200 and 10 denote here? Where might this key-value pair be stored?

Ans:

- [+2] Here, 200 is the content hash of the video, and 10 is Bob's node-id, the node where the video is stored
- [+1] The key-value pair will be stored on a node whose node-id is closest to the key 200. In this case, node-id 200 exists, so the key-value pair will be placed there

2. [4 points] Bob's drumming video gets popular enough that his friends Alice and Charlie offer to host a copy of the video on their nodes. First, Alice issues the command Put(200,1). Next, Charlie issues the command Put(200,2). Where will these key-value pairs be placed? Explain your answer.

Ans:

- [+1] <200,1> will be placed on node-id 160
- [+1] <200,2> will be placed on node-id 80
- [+1] Alice's request gets routed through node-ids 20, 40, 80, 160, 200 - node 200 is an exact match for the key, but it is full after Bob's request, so we go back to node 160 (the next non-full node in the route) and place the key-value pair there
- [+1] Charlie's request follows the same path but now even node 160 is full after Alice's request, so we go back to node 80 (the next non-full node in the route) and place the key-value pair there

3. [2 points] Diana hears about Bob's popular video and wants to see it. She issues the command Get(200) from her node (whose node-id is 10). What value(s) will she get back? Explain your answer.

Ans:

- [+1] Diana will get back value 2 from node 80

- [+1] This is because node 80 is the first node in the request's route that has information about key 200

Realtime & Multimedia [14 points]

TS Linux [4 points]

1.[4 points] Nathan, a TS-Linux system admin, observes that a time-sensitive application is experiencing occasional timing glitches despite being coded correctly using one-shot timers, doing admission control, and running at the highest priority level for scheduling.

Assume that Robert Love's lock-breaking mechanism is incorporated in the kernel to ensure that there aren't any long non-preemptible kernel code that would violate one-shot timer operation.

a) [2 points] Why could this be happening?

[+1] Priority Inversion

[+1] The app may be making blocking calls to a low-priority server (e.g. a window manager) which gets pre-empted by some higher priority task.

b) [2 points] What could be done to ensure that the glitch does not re-occur?

[+2] Modify the scheduler to inherit the priority of the calling application on blocking calls to system services.

PTS [10 points]

1. [4 points]

[4 points] Consider three channels:

- Ch1 contains items with timestamps: 100, 200, 300, 400, 500
- Ch2 contains items with timestamps: 140, 250, 370, 470
- Ch3 empty

Assume Get on a channel will return an item whose timestamp is closest to the TS requested. The returned item could have timestamp greater or lesser than TS if the requested TS is not available.

Two threads T1 and T2 execute concurrently:

T1:

<item1, ts1> = Get(ch1, 270);

```
<item2, ts2> = Get(ch2, ts1);
Digest = Process(item1, item2);
Put(ch3, Digest, (ts1 + ts2) / 2);
```

T2:

```
<item1, ts1> = Get(ch1, "oldest");
<item2, ts2> = Get(ch2, "now");
Digest = Process(item1, item2);
Put(ch3, Digest, ts2 - ts1);
```

What will be the timestamps of the two items in ch3 after both threads complete? Show all steps for each thread.

Ans:

- [+0.5] T1: Get(ch1, 270) returns ts1 = 300
- [+0.5] T1: Get(ch2, 300) returns ts2 = 250
- [+0.5] T1: Put uses (300 + 250) / 2 = 275
- [+0.5] T2: Get(ch1, "oldest") returns ts1 = 100
- [+0.5] T2: Get(ch2, "now") returns ts2 = 470
- [+0.5] T2: Put uses 470 - 100 = 370
- [+1] Ch3 will contain two items with timestamps: 275 and 370

2. [2 points] (Answer True/False with justification – No points without justification)

Two threads T1 and T2 simultaneously execute Put(ch1, item1, ts=100) and Put(ch1, item2, ts=100) respectively. PTS will reject one of the Put operations to maintain timestamp uniqueness.

Ans:

- [+1] False
- [+1] Both Put operations will succeed. PTS allows multiple items with the same timestamp in a channel.

3. [2 points] (Answer True/False with justification – No points without justification)

If thread T1 executes Put(ch1, item, ts=50) and thread T2 later executes Put(ch1, item, ts=50), T2's Put will overwrite T1's item in the channel.

Ans:

- [+1] False
- [+1] PTS channels are append-only and immutable. Items are never overwritten. Both items will coexist in the channel with the same timestamp.

4. [2 point] (Answer True/False with justification – No points without justification)

Once an item is moved to the garbage list of a channel, it cannot be recovered unless a pickling handler is configured for archival storage.

Ans:

- [+1] True
- [+1] Items in the garbage list are eventually discarded. Without a pickling handler to archive them, they are lost permanently.

Failures & Recovery [10 points]

RioVista [6 points]

1. [6 points] Vista is a recoverable memory library with a similar interface to Satya's LRVM (e.g. begin, end, abort), but runs assuming it is on top of the Rio persistent file cache. This assumption allows RioVista to simplify some operations when compared to RVM.

a) [2 points] What occurs when an application **begins** a transaction using RioVista? How does this differ from LRVM?

Ans:

- [+1] RioVista stores a before-image of the data to be modified in an undo log residing in the Rio file cache.
- [+1] In RVM the undo record is assumed to be in volatile memory, rather than a persistent file cache.

b) [2 points] What occurs when an application **ends** a transaction using RioVista? How does this differ from LRVM?

Ans:

- [+1] RioVista will simply delete the undo-log.
- [+1] RVM will, in addition to deleting the undo record, have to persist the updated data to a redo-log on disk.

c) [2 points] What occurs when an application **aborts** a transaction using RioVista? How does this differ from RVM?

Ans:

- [+1] RioVista will apply the undo-log to the external data segment (mapped into the Rio file cache) and delete the undo-log.
- [+1] LRVM will apply the undo record to the virtual memory region named in the set-range call.

Quicksilver [4 points]

1. [2 points] In the Quicksilver model, the node where the transaction originates is usually the default coordinator.

From a design perspective, why is it risky for a client node to remain the coordinator for a transaction involving a remote File Server and how does Quicksilver mitigate this risk?

Ans:

- [+1] Clients are often the most brittle parts of the system (may crash, users turn them off, etc.). If the coordinator is gone, the transaction tree's root is lost too.
- [+1] Quicksilver allows the client to delegate ownership to a more robust node (like the file server itself). This ensures that even if the client crashes, the robust node can drive the transaction to a conclusion

2. [2 points] A developer writing a simple text-editing application on Quicksilver decides to be extremely safe and issues a Log Force command after every single character keystroke.

Explain why this design decision negatively impacts other unrelated high-performance applications (e.g., a video renderer) running on the same node.

Ans:

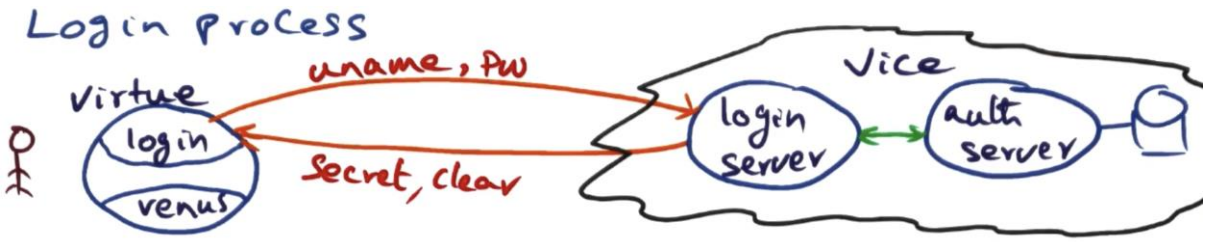
- [+1] The Log Manager maintains a single log containing modifications for all processes/clients running at that node, not just the text editor
- [+1] A Log Force triggers a synchronous I/O operation to persistent storage. By forcing frequently, the text editor forces the Transaction Manager to flush the entire in-memory log segment (including data from the video renderer) to disk, degrading the performance of all concurrent applications on that node

Security [5 points]

AFS [5 points]

1. [5 points]

Reference to the following diagram of the login process from a client.



a) [1 point] During the login process, is the password sent in clear text? How is it kept secure?

Ans: [+1] During the login process, the password is not sent in clear text. Login is a special case of bind where the password is used as the encryption key (i.e., HKC of the bind protocol).

b) [2 points] How does Venus get the key to use for the login session (HKC) from Vice's response?

Ans:

[+1] The cleartoken (encrypted with the PW) is a data structure that contains the HKC.
[+1] Venus can decrypt cleartoken to extract the key (HKC) for use in this login session.

c) [2 points] How does the AFS avoid overexposure of the client's username and HKC in future communication during the login session of the client?

Ans: [+1] The secrettoken -> ephemeral client id is used in place of the username in future requests to establish new RPC sessions during the login session.

[+1] After RPC session establishment, the SK is used in place of the HKC.

HIDDEN - Potpourri [13 points]

LRVM [10 points]

1. [10 points] Explain how each of the following LRVM features help improve the performance of a server built on top of LRVM:

a. [2 points] Use undo records to abort a transaction instead of loading the old data from disk.

Ans:

- [+2] Storing undo records in memory helps avoid reading old data from the disk when aborting a transaction, which can be slow.

b. [2 points] Store updates as logs instead of writing them directly to external data segments.

Ans:

- [+2] Writing directly to external data segments would require random disk writes, whereas writing to a log segment can be done sequentially, which is faster.

c. [2 points] Read from the tail of the redo log during truncation and crash recovery.

Ans:

- [+2] The redo log may contain updates to the same block of data from successive transactions. By reading from the tail, LRVM can avoid redundant work by applying only the latest update.

d. [2 points] Use the no-restore mode in begin_transaction.

Ans:

- [+2] Avoids the overhead of copying data and creating an in-memory undo record for the set-range calls within this transaction.

e. [2 points] Use the no-flush mode in end_transaction.

Ans:

- [+2] Reduces commit latency as a synchronous redo log flush is avoided

AFS [3 points]

1. [3 points] In general, private key (symmetric) encryption systems require O(N^2) keys, where N is the number of users.

a) [1 point] Why do symmetric private key systems require O(N^2) keys?

Ans: General private key systems are O(N^2) because there are distinct keys from all users to all users.

b) [2 points] Does AFS exhibit the same key scaling behavior? Justify your answer.

Ans:

[+1] No, AFS is O(N)

[+1] In AFS it is simply all users to one server, thus only O(N).